Computer Graphics: Lecture 14

Scenes

Welcome back!

Last time we learned about meshes.

[What is a mesh?]

We learned how to load a mesh from a file.

Specifically, an OBJ file.

We used this knowledge to draw a teapot.

This time

This time we're going to learn more about scenes.

You've seen them before: in our camera lecture, we used a simple scene node class to represent objects in our scene, including the camera.

But something was missing…

Real Scenes

Interactive software has more complex scenes…

That's a screenshot of the game Banjo-Kazooie. Banjo is the bear, Kazooie is the bird, and they are two separate scene objects.

Scene Hierarchy

Not only that, but for convenience in animation, one of the objects is subordinate to the other.

Specifically: when Banjo moves, Kazooie moves with him.

However, the reverse is not true: when Kazooie pops out of his backpack, Banjo is not moved.


Banjo-Kazooie

Scene objects

This pattern is more common than just for animating characters.

Frequently, 3D levels will be built out of chunks. For example, a building can be a collection of rooms. The rooms can have items in them:

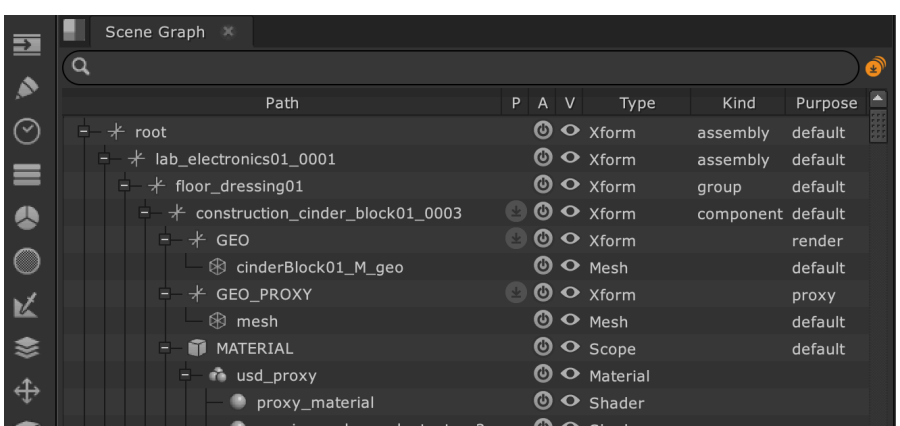
Hierarchy:

- Library
  - Foyer
  - Reading room 1
  - Reading room 2
    - Desk
      - Book

The scene hierarchy

If you have ever used a 3D engine before, you have probably been exposed to this hierarchy, whether you realized it or not.

Unity, for example, assumes that your scene will be in the form of a tree. So does Unreal, so does Godot.



An example scene graph from Unreal Editor.

The scene hierarchy (2)

Is this strictly, absolutely required?

No, we already saw that we could build a simple scene without it.

And furthermore, the GPU doesn't normally know or care about your hierarchy unless you tell it. Typically by the time we do our rendering, we just have a model matrix, a viewProj matrix, and maybe some textures and a vertex buffer.

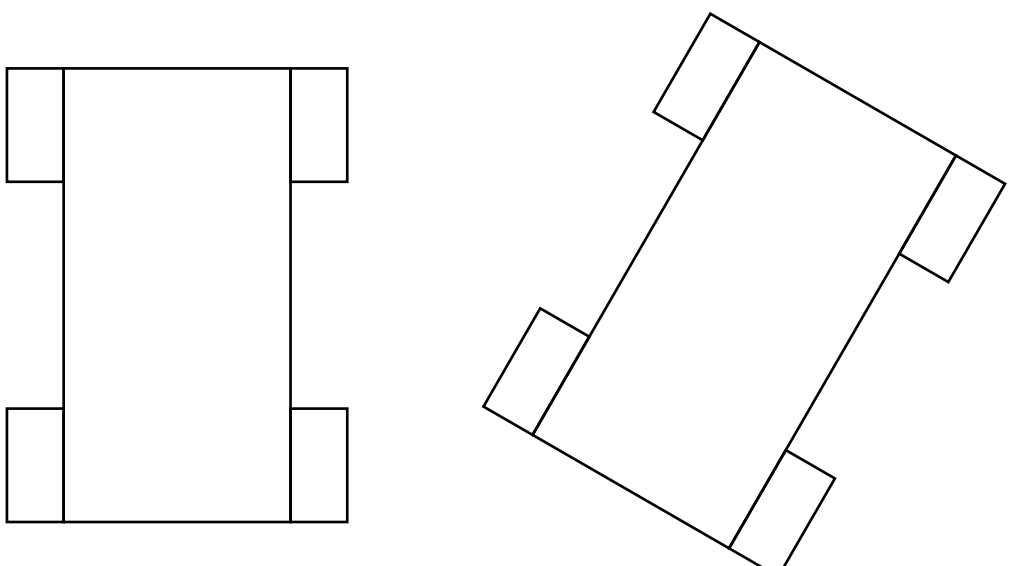
So we don't *have* to represent scenes hierarchically.

But we typically *want* to...

Consider a race car

Suppose we're making a racing simulator.

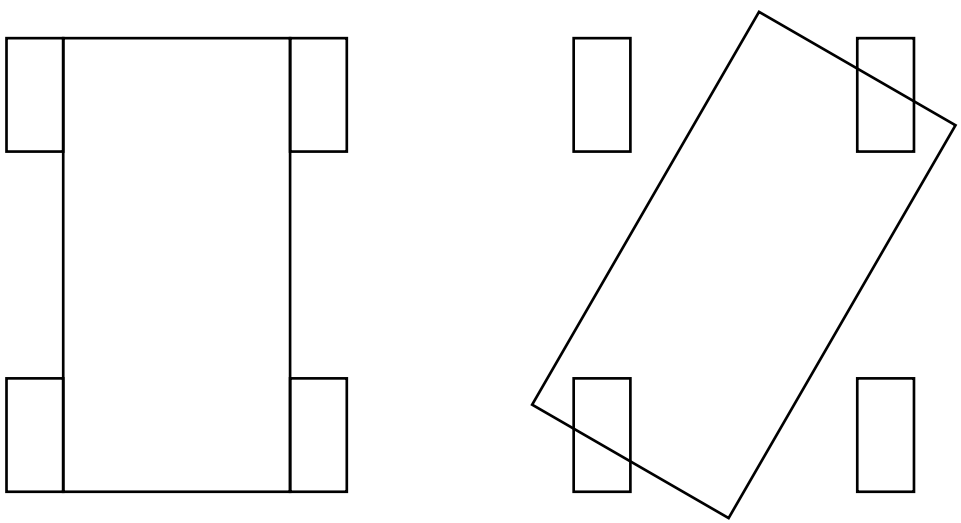
We have a car. The car has a chassis and four wheels…



This is what we want to happen when the car is rotated (left to right)

Consider a race car (2)

Now consider what happens with no hierarchy at all:



This is what happens if we don't use hierarchical grouping. We rotated the chassis, but the wheels weren't attached to it.

Consider a race car (3)

Is it possible to do this *without* a hierarchy?

Yes. You could move all the wheels independently.

However, this would mean first moving the wheels to the origin, then offsetting them by the correct amount, then rotating around the origin, then translating them to the final position of the car, so that they would be rotated and at the correct offset relative to the chassis.

It's just kind of a lot. Especially given that the hierarchy gives a useful way of designing scenes.

Designing Scenes

Remember that humans usually design game scenes.

We typically like to paint with a broad brush first, designing higher level areas, before zooming in with details.

This is analogous to the way drawing often works, or music composition, or even top-down software design. Start with big areas, subdivide into smaller details.

It might be worth exploring some scenes: NoClip Game scene Viewer

Questions?

Coding Scenes

So, how do we achieve this effect?

Here is the basic idea:

- Extend the idea of the scene node. Not only does a scene node have a matrix, it also has a parent and a list of children.
- Each scene node still has a matrix. However, this matrix is not *final*. It represents the scale, rotation, and position of a node **relative to its parent**.
- Then we have an actual final matrix. This matrix is constructed by applying the transformations from the parent (and its parent, and so on). This is the model matrix we send to the video card.

The basic idea

Suppose we have a simple solar system. Pretend the orbits are circular.

A planet rotates around a star. A moon rotates around the planet.

These are scene nodes. We want it to work like this:

- When the star is moved, the planet moves with it.
- When the planet is moved, the planet's moon moves with it.

We also want this to apply to rotation.

In fact, we want *every* transformation that applies to a parent to also apply to its children. But *not vice versa*.

The basic idea (2)

The question is: how do we accomplish this?

[suggested designs?]

Remember that each item in the scene has two matrices:

1. A local matrix, representing its changes relative to the parent.
2. A final matrix, representing it's model matrix that has incorporated all the changes, which we draw.

A simple hierarchical scene node

The basic idea is this: our goal is to generate the final marix. That is the matrix we need to send to the video card.

A node's local matrix represents its movement *relative to the parent*.

Therefore, we take the *final matrix of the parent*, and post multiply it by the *local matrix of the node*.

The result is the final matrix of the node.

Then, we propagate this change forward: the final positions of the node's children are their local matrices times the final matrix of the parent.

A simple example

Suppose the sun is at the center of the scene. It's position is 0, 0, 0. It's local and final matrices are both the identity matrix.

Suppose the planet starts at position 100, 0, 0, relative to the sun.

The sun is rotated by a quarter turn around the Z axis. What happens to the planet?

After the sun is rotated, its final matrix is:

$$\begin{pmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

A simple example (2)

The planet's local matrix is simply a tranlsation. It is offset from the sun:

$$\begin{pmatrix} 1 & 0 & 0 & 100 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

What happens when we apply the planet's local translation to the sun's final translation?

$$\begin{pmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 1 & 0 & 0 & 100 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

A simple example (3)

What does that mean? First, consider the position:

Rotating the sun caused the planet to rotate around it. Previously the planet was at (100, 0, 0). Now it is at (0, 100, 0), a quarter turn rotation.

But what's more, the planet's matrix also includes a rotation by a quarter turn in it.

That means the moon will also rotate a quarter turn and be translated, causing it to follow the planet. Just like the wheels of the car, before.

[whiteboard?]

Questions?

Coding it

Okay, let's code up a scene graph![1]

Let's try to think about what we need to write:

- A new node class that has the required two matrices and parent-child hierarchy. (A tree)
- A method that calculates the final matrix from the parent.
- New drawing code that recursively walks the scene.

Let's start with the node class…

[1] It's a tree, but in the literature this data structure is called a Scene Graph, because nodes can sometimes reference templates that end up being in more than one node. But let's think of them as trees.

The node class

```
export class SceneNode {
    finalMatrix: mat4 = mat4.create();
    children: SceneNode[] = [];
    parent: SceneNode | undefined;

    localMatrix: mat4 = mat4.create();

    finalBuf: GPUBuffer;
    modelBg: GPUBindGroup;

    mesh: LoadedMesh | undefined;
    name: string | undefined;
    ...
}
```

The name

The name is just an optional string.

We can use it for debugging, or for inspecting the scene.

We can also use it for identifying a particular node so we can apply a custom transformation to it.

This is done in our sample.

Otherwise, it's not strictly required. That said, most scene graphs seem to have nodes with meaningful names. It makes it easier to display them to the scene creator.

The node class fields

First, we store two matrices: `finalMatrix` and `localMatrix`.

These are `mat4`s, which means they are accessible to the CPU (they're in main memory).

However, there is also a `GPUBuffer` for the final matrix. And a bind group that refers to that buffer.

The shader will only see the matrix in that buffer, and it will be bound to the group and binding of the model matrix.

This is important: we don't want to do all our transformations for every single vertex. We want to combine them all into one matrix.

The methods

This scene node is a little simpler than the one we looked at in the camera controls lecture. We don't have yaw, pitch, or roll. We do everything with the matrix.

The methods act directly on this matrix:

```
move(pos: vec3): void {
    mat4.translate(this.localMatrix, this.localMatrix, pos);
}
rotateX(amount: number): void {
    mat4.rotateX(this.localMatrix, this.localMatrix, amount);
}
... // more methods
```

The constructor

The constructor is also similar to before. We create our matrix buffer and a bind group.

More importantly, the constructor also calls a method called `updateData`

```
constructor(
  device: GPUDevice,
  layout: GPUBindGroupLayout, // for the bind group
  name?: string // this is an optional parameter
) {
  this.name = name; // will be undefined if `name?` omitted
  this.finalBuf = ... // create a new buffer to store the matrix
  this.modelBg = ... // create a bind group with the buffer
  this.updateData(device); // compute and copy the final matrix over
}
```

updateData

This is where we send the data to the GPU. We compute our final matrix, by using the parent's final matrix and our local matrix.

```
updateData(device: GPUDevice): void {
    // we'll discuss the `??` operator in a second...
    const parentMatrix = this.parent?.finalMatrix ?? mat4.create();
    mat4.mul(this.finalMatrix, parentMatrix, this.localMatrix);
    // copy the data to the GPU
    device.queue.writeBuffer(
      this.finalBuf, 0, new Float32Array(this.finalMatrix));

    // update children recursively
    for (let child of this.children) {
        child.updateData(device);
    }
}
```

The `?.` and `??` operators

`??` is called the null-coalescing operator.

It checks the left operand to see if it's either `null` or `undefined`.

If it is, the result of the expression is its right operand.

If not, the left operand is returned.

```
this.parent?.finalMatrix ?? mat4.create();
```

Will first determine if the parent exists. That's what `?.` is for.

It would be an error to access a field of a null/undefined value.
`null.finalMatrix` is not meaningful

The `?.` and `??` operators (2)

If the node's parent is `undefined`, then `this.parent?.finalMatrix` will be `undefined`, because the `?.` operator will short-circuit. It won't try to evaluate the rest.

Then, the `??` operator will kick in. That `undefined` value will cause `undefined ?? mat4.create()` to result in `mat4.create()`, which is the identity matrix.

All so this will happen: if we don't have a parent, treat the parent's matrix as if it were the identity. This means that we don't need a special case for the root of the sceen.

The recursive step

Once we determine the parent's matrix (or the identity matrix if there isn't one), we multiply it with the local matrix.

```
mat4.mul(this.finalMatrix, parentMatrix, this.localMatrix);
```

The result is the `finalMatrix`.

We apply this same logic to the children, recursively:

```
for (let child of this.children) {
    child.updateData(device);
}
```

The hierarchy

But where did children get added to the scene? Let's consider how we can add new children *below* a node in the hierarchy:

```
addChild(
  device: GPUDevice,
  layout: GPUBindGroupLayout,
  name?: string
): SceneNode
{
    const child = new SceneNode(device, layout, name);
    child.parent = this;
    this.children.push(child);
    return child;
}
```

The hierarchy (2)

In this case, the children are stored in an array.

This doesn't have to be the case: we could have required names, and used a hashmap to store the children. An array is fine for our purposes, however.

We walk the array of children in two circumstances:

1. When we need to update the matrix of our node and its children.
2. When we need to draw the node and its children.

Doing everything with a matrix

You might notice that there aren't yaw, pitch, roll, scale, etc.

That's because I decided to do everything with the matrix to demonstrate a matrix camera.

I kind of regretted it: the issue with only having the matrix is that all transformations modify the current state. It's easy to *rotate* by 90 degrees, but it's hard to *set the rotation to be* 90 degrees exactly, if that makes sense.

A raw matrix also has issues with stability: over time, the basis vectors will start to drift. For cameras, this will result in weird skewness creeping in the more you rotate.

Questions?

The Sample

`Sample11` has an example. It contains a scene hierarchy that draws a bunch of boxes.

One child box is scaled every frame. Notice how its children gets scaled.

All the boxes are rotated, and the child boxes are rotated additionally.

When the root node moves, notice that every box moves with it…

The root node

Scenes can be forests or trees.

If a scene is a tree, then it has a root node.

This is useful, because anything we do to the root node will end up happening to the entire scene.

[Can anyone think of an example of when this would be useful?]

Mirrored tracks

Consider this game:

Mario Kart 64

This game had *mirrored tracks.*

Mirrored tracks

A mirrored track is a track that is reflected about one axis.

If we reflect the track about the Y axis, and also

The scene can store a lot of things...

Alternatives to recursive drawing

Homework and Project